

1.4.2 : Three Sum



Return n (n = sum of 3 element = 0)
 a.sort();
 for (int i = 0; i < a.length; i++)

for (int j = i + 1; j < a.length; j++)

sum = a[i] + a[j] + a[k];

count += binarySearch(0, a.length - 1, a, -sum);

return count;

binarySearch (left, right, a, target)

while (left < right)

mid = (left + right) / 2

if (a[mid] == target)

return 1

else if (a[mid] > target)

return binarySearch(left, mid - 1, a, target);

else

return bS (mid + 1, right, a, target);

2. Two Pointers (int) and

Arrays.sort(a)

for (int i = 0; i < a.length; i++)

left = i + 1;

right = a.length - 1;

while (left < right)

sum = a[left] + a[right];

```

if (sum == -a[i])
    left = 1; count++; left++;
else if (sum < -a[i]) {
    left = 1;
}
}
right = 1;
}
return count;
    
```

1.4.5

- $N+1 \sim N$
- $1+1/N \sim 1$
- $(1+1/N)/(1+2/N) \sim 1$
- $2N^3 - 15N^2 + N \sim 2N^3$
- $\lg(2N)/\lg(N) = (\lg 2 + \lg N)/\lg N \sim 1$
- $\lg(N^2+1)/\lg N \sim \lg(N^2)/\lg N \sim 2 \lg N / \lg N \sim 2$
- $N^{100}/2^N \sim 0$

1.4.6

a) $n = N$ khi bắt và bị chứa đơn ở vế phải, vế sau chạy từ $i = 0 \rightarrow i \leq n$

$\rightarrow$ Vế trái chạy cùng mức, sum thay đổi.

$$N + \frac{N}{2} + \frac{N}{4} + \dots + 1 \rightarrow \text{Tổng này tiến dần về } 2N \text{ khi } N \rightarrow \infty$$

b) $\Theta(N)$ tăng dần

Vì từ vế trái bắt từ $i = 1$ và đi về phải lần lượt $i \leq N$. Vế trái tăng dần, sum tăng dần. $1 + 2 + 3 + \dots + N = \frac{N(N+1)}{2} \sim 2N^2$

$$\Rightarrow \Theta(N^2)$$

1.4.8: Counting the number of pair has the same NO Equal Pairs $\text{Cnt}[a]$. $O(N \log N)$
 Arrays sort(a).
 int total = 0;

n = a.length - 1;

for (int i = 0; i < n - 1; i++) {
 if (int tmp = 0;

while (i < n and a[i] == a[i + 1])
 tmp++;
 i++;

if (~~tmp~~ (tmp > 0))
 total += (tmp * (tmp - 1)) / 2;

return total;

1.4.10:

update Binary Search

If (a[mid] == key)

res = mid; // temporarily saving
 right = mid - 1; (continue finding)

1.4.12:

CommonElement (int[] a, int[] b).

if (a.length - 1 < b[0]) // b.length - 1
 a[0];

return;

int i = 0; int length;

int j = 0;

while (i < n and j < n)

if (a[i] < b[j])
 i++;

else if (a[i] > b[j]) j++;



else {

print(a[i] + " ")

int tmp = a[i]

while (i < n and tmp == b[j]) j++

while (i < n and tmp == a[j]) j++

}

1.4.13

Object overhead

reference : 16 bytes for every obj

primitives type : 8 bytes for a ref variable to other obj
int (4 bytes), double (8 bytes)
boolean (1 bytes)

padding

Array : 24 bytes round up to a multiple of 8 bytes
+ 4 bytes overhead (16 bytes + 8 bytes length + 4 bytes padding)

e) Accumulator

int N (4) ; double total (8 bytes)

→ 16 + 4 + 8 = 28

→ 8 (padding) = 28 + 4 = 32 bytes

f) Transaction

String who (8 bytes ref), data when (8 bytes ref)
double amount (8 bytes)

16 + 8 + 8 + 8 = 40 (not need to padding)

c) FCSOS

Stack Obj : 16 + 4 + 8 (ref to array String s3a)
= 28 + 4 = 32

String [] : (246 + 8 * C) bytes : C (element)

d) Point2D :

16 + 8 + 8 = 32

e) Integer valid : 32

g) Double (Wrapper)

primitive type : double : 8 bytes
16 + 8 = 24 bytes

Success is the sum of small efforts, repeated

1.4.14 (Hard)

Algorithm for 4-Sum

Idea: 4-Sum $\rightarrow$ 2-Sum and Two-pointers
list will save obj Pair involve: Sum of all
index i , index j

$\rightarrow$ target $= (X_1 + X_2)$

$\rightarrow$ Two pointer find Target

If found: Checking i, j, k, l has different?
if yes $\rightarrow$ Accept 4 value above

$\rightarrow O(N^2 \log N)$ because array N size ~~create~~
 $\frac{N^2}{2}$ pairs $\rightarrow$ So Arrays.sort() spend $N^2 \log(N^2)$

$$\approx N^2 \log(N)$$

Week 1

1) Test case : $n=3$, $\text{union}(0,1)$, $\text{union}(0,2)$
 Khởi tạo $\text{leader}[3] = [0,1,2]$
 $\text{union}(0,1)$

$\rightarrow [1,1,2] \rightarrow 2 \text{ set } \{0,1\} \text{ và } \{2\}$

$\text{union}(0,2)$

$i=0$: $[2,1,2]$

$i=1$: so sánh : $\text{leader}(1) == \text{leader}(0)$

$\forall i$: $\text{leader}(p)$ bị thay đổi thành $\text{leader}(q)$ tức

đó $\rightarrow \text{false}$ / ~~is~~

$\rightarrow [2,1,2] (\times)$

Đúng : $[2,2,2] (\checkmark)$

2) a) 1 tập hợp duy nhất.

b) Số lần cập nhật mỗi $\text{union}()$ là quick-find :

$$1+2+3+\dots+n-1 = \frac{(n-1+1) \times (n-1)}{2}$$

$$= \frac{n(n-1)}{2} \sim \frac{1}{2}n^2$$

c) $\text{find}(\text{int } i)$

while $(i \neq \text{id}[i]) \quad i = \text{id}[i]$

return i

TT $\text{find}(0)$ thành $n-1$ l + truy cập mảng

$\rightarrow \bigoplus$ Big Theta

d) Thay đổi $\text{parent}(q)$ khi cây size

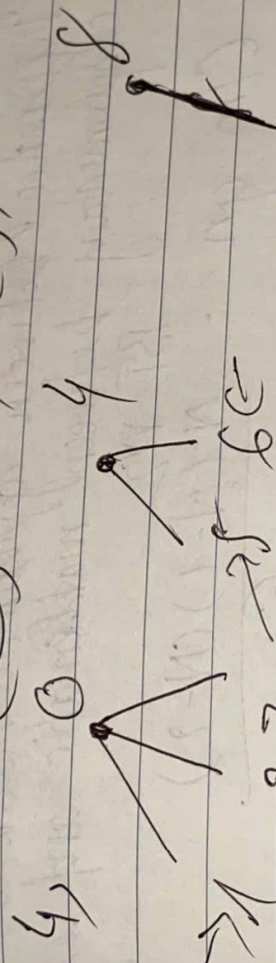
$\rightarrow$ mỗi nút từ $1 \rightarrow n-1$ lần của 0 có

đ. sâu $1 \dots \rightarrow \text{find}(0)$: đ. 0 là gốc

$\rightarrow 0 = \text{id}[0] \rightarrow \bigoplus$

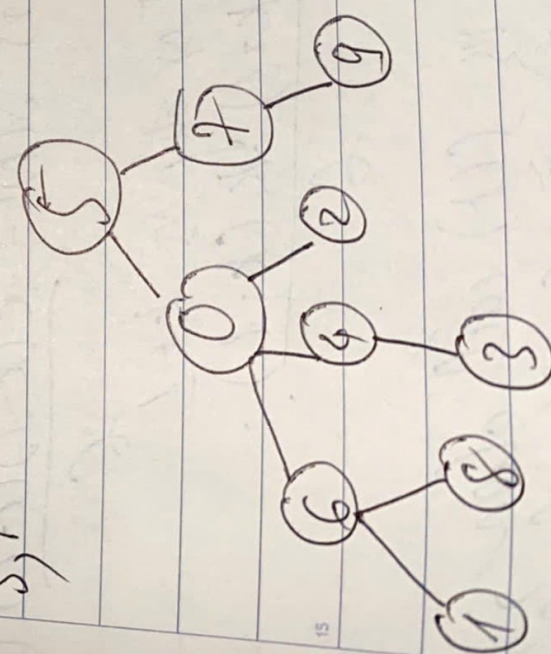
Inspire the love of learning

3, (4,0), (2,3), (5,0), (8,9), (4,5), (8,9)



gốc 2, 3, 5, 7, 10, 15
 con hoặc gốc 9
 parent[8] = {0, 4, 8, 9, 3, 5}
 lệnh union với gốc 9

0 thì là WQU



Bài 2: Phân tích thuật toán:

1.4.1 C/m qn: $\frac{N(N-1)(N-2)}{6}$

Bé có số voi $N=3 \rightarrow \frac{3 \cdot 2 \cdot 1}{6} = 1$ cách chọn

$\rightarrow$ CT đượ vs $N=3$

CT quy nạp: CT đượ vs $S(k) = \frac{k(k-1)(k-2)}{6}$ (k23)

Bé quy nạp: C/m CT ex đượ vs $k+1$

$$f(k+1) = \frac{(k+1) \cdot k(k-1)}{6}$$

Gi/s đượ có k pth vs $S(k)$ cách chọn, khi thêm pth $k+1$ vào $\rightarrow$ pth này có số cách đượ vs 2 trong k pth cũ là $\frac{k(k-1)}{2}$ cách

$$\rightarrow S(k+1) = S(k) + k^2$$

$$= \frac{k(k-1)}{2} + \frac{k(k-1)(k-2)}{6}$$

$$= \frac{3k(k-1) + k(k-1)(k-2)}{6}$$

$$= \frac{k(k-1)(k+1)}{6} \quad (\text{apcm})$$